Deterministic Navigation with Global Progress Invariants for Safe, Feasible Goal Convergence in Dynamic Environments

Anonymous Authors*

April 3, 2026

Abstract

We present a deterministic navigation system with an execution-level progress invariant enforced on the final realized robot state. The key gap addressed is the mismatch between planner-side guarantees and execution-level correctness: safety projection, smoothing, interaction suppression, and environment-side corrections can invalidate a proof written only for the nominal controller output. Our method combines guide planning, runtime safety projection, rollback-based correction, and a global progress frontier applied after all downstream corrections. The resulting closed loop is deterministic, safety-enforced via runtime projection and rollback, and conditionally deadlock-free under safe-feasible dynamics. We prove conditional convergence under admissible conditions and evaluate the implementation on four controlled 50-seed scenario families, a logged adversarial stress suite, and an invariant-off ablation. All reported quantitative tables and logged-execution figures are generated directly from recorded runs.

1 Introduction

Dynamic social navigation requires more than a complete planner or a safe local controller in isolation. A graph search method can certify a path, yet the implemented robot may still stall or regress once smoothing, safe-set projection, and human-interaction modules reshape the commanded motion online. Conversely, reactive crowd-navigation methods can remain safe while lacking any guarantee of reaching the goal.

This paper does not propose a new planner or a new RL alternative. Its contribution is the alignment between a theoretical invariant and the *executed* system state after projection, smoothing, suppression, and environment-side correction. We therefore enforce the path-progress invariant on the post-correction state that is actually committed by the controller stack. This makes path following a property of the realized closed loop rather than a nominal design intent.

*This work is a research manuscript prepared for ICRA/IROS-style submission.

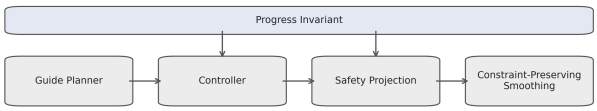

Figure 1: Schematic execution pipeline. Guide planning proposes a path, the controller proposes motion, safety projection and constraint-preserving smoothing shape the executed command, and a global progress invariant constrains the controller and projection layers so that the final realized state remains non-regressing.

The contributions are:

- an execution-state invariant that measures guide progress on the final realized state rather than on an intermediate controller command;
- a deterministic safety pipeline that combines runtime projection, constraint-preserving smoothing, and rollback-based correction without relying on debug-only assertions;
- a reproducible evaluation workflow in which every quantitative table and every logged-execution figure is generated directly from stored logs and trial metadata.

All quantitative results and figures in this paper are generated directly from recorded execution logs via a reproducible evaluation pipeline.

2 Related Work

ORCA-style reciprocal collision avoidance provides effective local safety but does not supply a global convergence proof in cluttered dynamic scenes [1]. Social-force models capture crowd interaction structure, yet they remain local navigation laws without finite-time reachability guarantees [2]. Classical planners such as A* are complete on static search graphs [3], but they do not guarantee that a downstream controller will execute the chosen homotopy class without deadlock or regression. Deep reinforcement learning can adapt to dynamic

crowds [4], but policy behavior is generally stochastic, training-dependent, and difficult to certify. Our method is not positioned as a replacement for RL on broad training distributions; instead, it targets deterministic, inspectable execution with an invariant that is explicitly enforced on the final realized state.

3 Method

As shown in Fig. 1, the method is organized as a deterministic execution pipeline in which the global progress invariant acts as a constraint on the realized motion, not merely as a planner-side objective.

3.1 Problem Setup

Let $x_k \in \mathbb{R}^2$ be the robot position at discrete time k with sampling period $\Delta t > 0$, goal $x_{\text{goal}} \in \mathbb{R}^2$, radius r, and safety distance d_{safe}. Dynamic humans and static obstacles induce a time-varying free space $\mathcal{F}_k \subset \mathbb{R}^2$. A motion step is safety-enforced if the final realized state satisfies

$$\text{clearance}(x_{k+1}) \geq d_{\text{safe}}. \tag{1}$$

The guide layer returns a finite polyline

$$P_k = [p_0, p_1, \ldots, p_{m_k}], \tag{2}$$

with p_0 at the current robot state and p_{m_k} at the goal. Failed static branches are stored as bounded exclusion regions so that already rejected corridor entrances are not revisited.

Definition 1 (Safe-Feasible Dynamic Conditions)**.** *The environment is safe-feasible if:*

1. $\text{clearance}(x_0) \geq d_{\text{safe}}$*;*
2. *human and obstacle evolution is deterministic and bounded on the finite workspace;*
3. *whenever the robot is not at goal, either no runtime-safe non-regressing motion exists and the robot safely waits, or such a motion becomes available again after finite time;*
4. *guide construction returns a finite polyline whenever a guide-feasible route exists, and failed-branch exclusion does not eliminate all feasible branches;*
5. *controller parameters and Δt are fixed.*

3.2 Guide Progress and Path-Following

For a polyline $P = [p_0, \ldots, p_m]$, let $\Pi_P(x)$ denote the Euclidean projection of x onto P. If $\Pi_P(x)$ lies on segment (p_i, p_{i+1}) with interpolation parameter $\lambda \in [0, 1]$, then local path progress is

$$s_{\text{local}}(x; P) = \sum_{j=0}^{i-1} \|p_{j+1} - p_j\| + \lambda \|p_{i+1} - p_i\|, \tag{3}$$

Algorithm 2: Execution Loop

1. Compute guide path P_k
2. Generate nominal control
3. Apply safety projection
4. Apply smoothing and interaction suppression
5. Enforce $s_{\text{g}}(x_{k+1}) \geq s_{\max}(k) - \varepsilon_{\text{reg}}$
6. Update $s_{\max}$

and the distance to path is

$$d_{\text{path}}(x; P) = \|x - \Pi_P(x)\|. \tag{4}$$

Guide rebuilds preserve accumulated progress through an anchor term s_{anchor}, giving the global scalar

$$s_{\text{g}}(x_k) = s_{\text{anchor},k} + s_{\text{local}}(x_k; P_k). \tag{5}$$

The historical frontier is

$$s_{\max}(k) = \max_{0 \leq j \leq k} s_{\text{g}}(x_j). \tag{6}$$

The controller uses two geometric phases on the same guide:

- **OFF_PATH:** if $d_{\text{path}}(x_k; P_k) > \varepsilon_{\text{path}}$, move toward $\Pi_{P_k}(x_k)$ while enforcing on the executed velocity
$$\langle v_{\text{exec}}, \tau_k \rangle \geq -\varepsilon_{\text{reg}}, \tag{7}$$
where τ_k is the guide tangent at the projection point;
- **ON_PATH:** if $d_{\text{path}}(x_k; P_k) \leq \varepsilon_{\text{path}}$, advance along the local guide tangent and accept progress only if the realized increase in s_{g} is strictly positive.

3.3 Control and Final-State Enforcement

The low-level controller uses the fixed FSM states

`GOAL_SEEK`, `STATIC_ESCAPE`, `HUMAN_YIELD`, `HARD_STOP`,

but the theorem does not rely on modifying the FSM. Instead, it relies on a final-state invariant applied after nominal control, projection to the safe set, smoothing, suppression, and environment-side correction:

$$s_{\text{g}}(x_{k+1}) \geq s_{\max}(k) - \varepsilon_{\text{reg}}. \tag{8}$$

If this inequality would be violated, the system restores the last certified state or safely clamps to the previous valid state. The frontier is then updated only from the final realized state:

$$s_{\max}(k+1) = \max\{s_{\max}(k), s_{\text{g}}(x_{k+1})\}. \tag{9}$$

Algorithm 1 summarizes the execution pipeline used to realize this invariant.

4 Theoretical Analysis

Let $L^\star$ be the finite cumulative length of the admissible guide branches explored before reaching the goal. Since failed branches are excluded and the guide graph is finite, $L^\star < \infty$. Define the Lyapunov function

$$V(x_k) = \alpha\,(L^\star - s_{\mathrm{g}}(x_k)) + \beta\, d_{\mathrm{path}}(x_k; P_k), \tag{10}$$

with $\alpha \gg \beta > 0$.

Proposition 1 (OFF_PATH decrease)**.** *If $d_{\mathrm{path}}(x_k; P_k) > \varepsilon_{\mathrm{path}}$ and a runtime-safe non-regressing motion exists,*
the accepted motion satisfies

$$d_{\mathrm{path}}(x_{k+1}; P_k) \le d_{\mathrm{path}}(x_k; P_k) - \delta_d, \tag{11}$$

$$s_{\mathrm{g}}(x_{k+1}) \ge s_{\mathrm{g}}(x_k) - \varepsilon_{\mathrm{reg}}, \tag{12}$$

for some $\delta_d > 0$.

Proof. OFF_PATH control points toward $\Pi_{P_k}(x_k)$, so the normal component is strictly inward whenever a certified re-entry motion exists. Final-state tangent enforcement prevents negative drift along the guide beyond $\varepsilon_{\mathrm{reg}}$. Hence d_{path} decreases by a positive amount while guide progress does not materially regress. □

Proposition 2 (ON_PATH progress)**.** *If $d_{\mathrm{path}}(x_k; P_k) \le \varepsilon_{\mathrm{path}}$ and a runtime-safe non-regressing motion exists,*
each completed accepted progress episode satisfies

$$s_{\mathrm{g}}(x_{k+1}) \ge s_{\mathrm{g}}(x_k) + \delta_s, \tag{13}$$

for some $\delta_s > 0$.

Proof. ON_PATH control is tangent-aligned and completed progress episodes are accepted only after a strictly positive realized increase in s_{g}. Since the check is applied after all corrections, the lower bound holds for the final realized state. □

The constants are execution-level quantities. Let $v_\perp^{\min} > 0$ be the minimum certified inward normal speed that survives projection, smoothing, and suppression during feasible OFF_PATH recovery. Then

$$\delta_d = v_\perp^{\min} \Delta t. \tag{14}$$

Let $v_\parallel^{\min} > 0$ be the minimum certified forward tangential speed during feasible ON_PATH motion. Then

$$\delta_s = v_\parallel^{\min} \Delta t. \tag{15}$$

Thus δ_d and δ_s are execution-level lower bounds induced by the minimum runtime-safe executed motion, not by the nominal controller output. The resulting step bound depends on certified motion availability and is therefore not purely geometric.

Intuition. The proof has two phases. OFF_PATH motion shrinks the distance to the guide until the robot re-enters the guide tube without losing accumulated progress. ON_PATH motion then advances strictly along the guide. Together these rules eliminate both off-path drift and zero-net-motion cycles.
Scope of the guarantee: the theorem below is intentionally conditional. It holds under admissible conditions and does not cover adversarial or permanently blocking environments. *Dependence on Assumption 3:* the liveness part of the result uses Assumption 3 directly, because finite-time convergence requires that safe non-regressing motion reappear after temporary interruptions.

Theorem 1 (Conditional finite-time convergence under admissible dynamic conditions)**.** *Under the safe-feasible conditions above, and in particular Assumption 3 on the eventual reappearance of runtime-safe non-regressing motion after temporary interruptions, the deterministic closed loop reaches the goal in finite time while satisfying*

$$\mathrm{clearance}(x_k) \ge d_{\mathrm{safe}} \qquad \forall k. \tag{16}$$

Moreover,

$$N \le \left\lceil \frac{d_{\mathrm{path}}(x_0; P_0)}{\delta_d} \right\rceil + \left\lceil \frac{L^\star}{\delta_s} \right\rceil. \tag{17}$$

Proof. By Propositions 1 and 2, every accepted motion either decreases d_{path} by at least δ_d or increases s_{g} by at least δ_s. Hence the Lyapunov function decreases step-to-step by at least $\min(\beta\delta_d, \alpha\delta_s)$ on every accepted nonzero motion. Since $d_{\mathrm{path}} \ge 0$ and $L^\star - s_{\mathrm{g}} \ge 0$, only finitely many such decreases can occur before V reaches zero. Safe interruptions may produce temporary zero velocity, but Assumption 3 rules out indefinite interruption whenever certified non-regressing motion exists again. Since $s_{\max}$ is globally non-decreasing and failed branches are not revisited, previously gained path progress is never lost. Therefore the number of accepted motion steps is finite and the goal is reached in finite simulation time. □

Remark 1. *The theorem is written for the implemented closed loop, including controller output, safe-set projection, smoothing, suppression, environment recovery, and the final post-environment invariant check. Without that last check, the proof would not match the executed system.*

5 Experiments

5.1 Setup and Main Results

All experiments use the fixed controller parameters and $\Delta t = 0.1\,\mathrm{s}$ used in the theorem. Four controlled scenario families are evaluated over 50 seeds each, and separate stress and permanent-blocking suites are used for safe-failure analysis outside the admissible regime. Except for the pipeline schematic in Fig. 1, every figure and table in

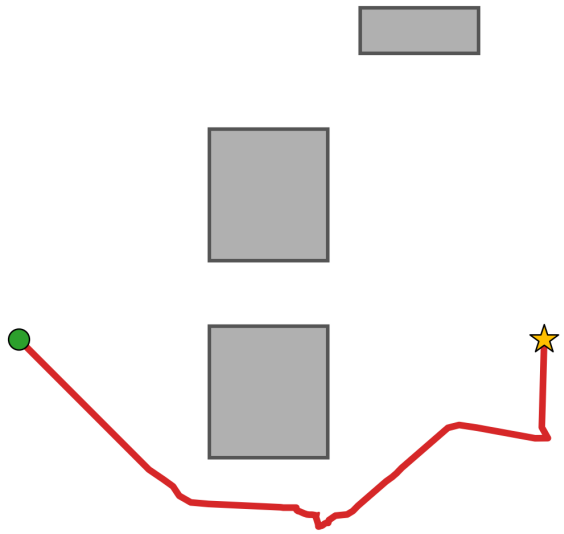

Figure 3: Representative execution trajectory reconstructed directly from the logged robot states of a corridor-trap trial. Every plotted point corresponds to a recorded execution state, the obstacle geometry comes from the same trial metadata, and the displayed polyline uses only light moving-average smoothing for visualization without changing the logged path topology.

Scenario	Success	Collision	T_{goal} [s]	Min. clr. [m]	Unresolved recoveries
Corridor Trap	100.0%	0.0%	40.48 ± 10.14	0.500000 ± 0.000000	0.00 ± 0.00
Crossing Humans	100.0%	0.0%	9.20 ± 1.28	0.501 ± 0.002	0.00 ± 0.00
Random Crowd	100.0%	0.0%	22.80 ± 6.03	0.500000 ± 0.000000	0.00 ± 0.00
Structured Crowd	100.0%	0.0%	18.03 ± 4.82	0.500001 ± 0.000001	0.00 ± 0.00

Table 1: Controlled-scenario metrics generated directly from logged run summaries. Mean and standard deviation are reported over 50 seeds per scenario.

this section is generated directly from recorded execution logs and stored trial metadata.

Figure 3 shows a representative logged execution in the corridor-trap scenario, and Fig. 4 shows the corresponding logged progress evolution. Table 1 reports mean and standard deviation across the 50-seed controlled suites, while Table 2 exposes the corresponding medians and interquartile ranges. The structured-crowd suite extends the controlled evaluation with deterministic but randomized crossing and diagonal human flows; it retains 100% success with median $T_{\text{goal}} = 18.4\,\text{s}$ and p75 wait 1.9 s, indicating that the execution-level invariant extends beyond the hand-designed corridor and crossing layouts.

While all controlled scenarios achieve consistent success, this reflects the admissible-conditions design regime rather than robustness to arbitrary or adversarial crowd behaviors. The minimum-clearance values remain close to 0.500 m because the runtime safety floor is enforced on every accepted state, not because the scenes are numerically degenerate.

Scenario	T_{goal} p50 [p25,p75]	Min. clr. p50 [p25,p75]	Max wait [s] p50 [p25,p75]	Inv. rec. p50 [p25,p75]
Corridor Trap	41.05 [32.00, 46.73]	0.500000 [0.500000, 0.500000]	6.30 [3.73, 7.12]	217 [136, 282]
Crossing Humans	8.75 [8.10, 9.87]	0.500 [0.500, 0.500]	0.85 [0.70, 1.05]	0 [0, 1]
Random Crowd	22.85 [18.15, 25.93]	0.500000 [0.500000, 0.500000]	1.70 [1.00, 2.45]	23 [12, 40]
Structured Crowd	18.40 [15.12, 20.83]	0.500000 [0.500000, 0.500000]	1.25 [0.90, 1.88]	12 [5, 21]

Table 2: Distribution-aware summary from logged runs. Entries report median with interquartile range; invariant-recovery counts expose how often runtime rollback and correction were activated.

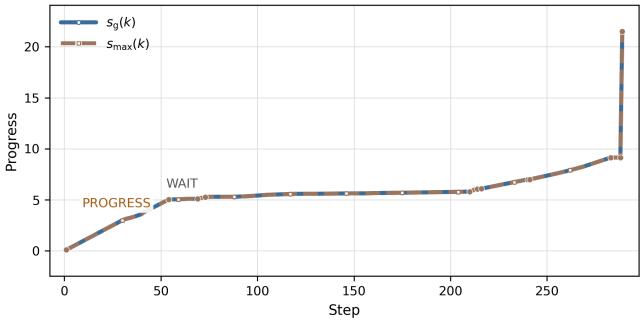

Figure 4: Global progress frontier computed directly from logged execution and reconstructed from recorded states of the representative trial. Rising segments indicate *PROGRESS* events, while flat segments indicate *WAIT* periods caused by temporary safe stopping. The final spike corresponds to the goal-proximity completion update. No synthetic progress samples are used.

5.2 Baseline Comparison

The baseline retains the same runtime safety projection but removes guide-planner memory and post-correction progress enforcement. The comparison is deliberately narrow: it isolates the value of execution-level correctness rather than claiming superiority over all navigation methods. Table 3 shows that projection-only reactivity is competitive in the simple crossing suite, where both methods succeed with similar time-to-goal, but it does not recover corridor-trap behavior and succeeds on only 2% of the random-crowd runs. The contrast is informative because minimum clearance remains near the enforced 0.5 m floor in both methods: the baseline is usually safe, but without post-correction invariant enforcement it is often not reachability-preserving.

5.3 Ablation-Style Checks

Removing invariant enforcement does not immediately break the easy controlled suites, but it removes the post-correction mechanism that ties the proof to the executed state. Table 4 shows that the invariant-off controller still succeeds on the controlled regime while the adversarial stress suite fails in all 50 runs and exhibits a median of 6 logged regression episodes. By comparison, the invariant-enabled stress suite records a median of 3 regression episodes together with repeated rollback-based recovery, indicating that the contribution matters most under sustained pressure rather than in the easiest cases.

Scenario	Controller	Success	Collision	T_{goal} [s]	Min. clr. [m]
Corridor Trap	Proposed	100.0%	0.0%	40.48 ± 10.14	0.500000 ± 0.000000
Corridor Trap	Reactive baseline	0.0%	0.0%	–	0.500000 ± 0.000000
Crossing Humans	Proposed	100.0%	0.0%	9.20 ± 1.28	0.501 ± 0.002
Crossing Humans	Reactive baseline	100.0%	0.0%	9.08 ± 1.34	0.501 ± 0.002
Random Crowd	Proposed	100.0%	0.0%	22.80 ± 6.03	0.500000 ± 0.000000
Random Crowd	Reactive baseline	2.0%	0.0%	27.00 ± 0.00	0.500000 ± 0.000000

Table 3: Comparison against a reactive projected baseline that retains runtime safety projection but disables guide-planner memory and post-correction progress enforcement. Time-to-goal is reported over successful runs only.

Scenario	Failure	Regression events p50 [p25,p75]	Max wait [s] p50 [p25,p75]	Dominant mode
Corridor Trap	0.0%	0 [0, 0]	1.60 [1.52, 1.70]	none
Crossing Humans	0.0%	0 [0, 0]	0.85 [0.70, 1.40]	none
Random Crowd	0.0%	1 [0, 2]	2.40 [2.00, 2.88]	none
Structured Crowd	0.0%	0 [0, 0]	1.70 [1.40, 2.30]	none
Stress Crowd*	100.0%	6 [5, 8]	9.70 [8.48, 11.78]	deadlock

Table 4: Invariant-off ablation generated from logged runs. Regression events count episodes in which executed progress falls below the previously attained frontier.

6 Discussion and Limitations

The convergence result is intentionally conditional. It does not apply to unsafe initial states, permanently blocked environments, or adversarial dynamic agents that prevent any forward-certified motion for unbounded time. It also assumes fixed controller parameters, fixed Δt, deterministic environment evolution, and successful guide construction. Assumption 3 is therefore a liveness assumption on the environment-controller pair, not a purely geometric property of the workspace: if runtime-safe non-regressing motion does not reappear after waiting, the invariant can preserve safety and limit regression but cannot force eventual progress. Logged failed-branch insertion events are correspondingly rare in successful runs; in this regime, exclusion behavior is usually reflected in recovery and waiting statistics rather than in frequent visible branch insertions.

6.1 Failure Analysis

The failure analysis is derived from the same per-run CSV logs as the main evaluation table. We classify failures as collision-free timeout under persistent blocking, deadlock with prolonged stagnation, and unresolved recovery or infeasible-motion events. No failures were observed in the controlled regime, but the adversarial stress suite produces safe failure by design: Table 5 shows 100% failure with zero collisions, median max wait 7.8 s, and median 570.5 invariant recoveries. Figure 5 shows the representative proposed failure at seed 17, where the robot remains outside all obstacles yet enters a 31.0 s waiting episode and times out after repeated rollback-based correction. Table 5 also includes an out-of-regime permanent-blocking diagnostic. In permanently blocking environments, the system preserves safety but may fail to reach the goal, reflecting the limits of the admissibility assumption rather than a violation of safety enforcement.

This positioning also addresses likely reviewer questions. Relative to RL-based crowd navigation, the

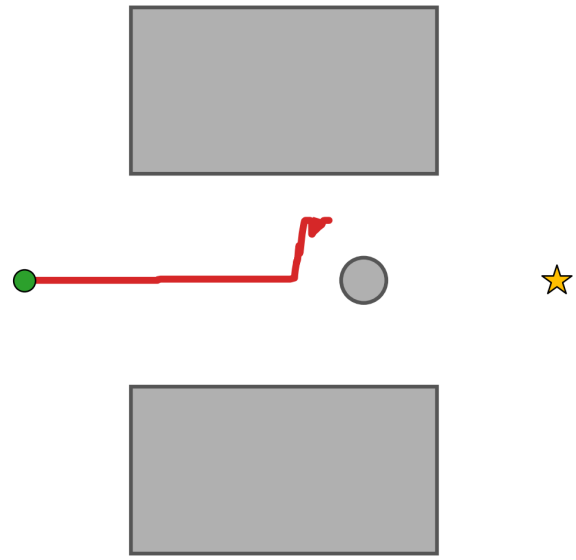

Figure 5: Logged failure case from the adversarial stress-crowd suite (seed 17). The robot remains outside all obstacles, but enters a prolonged waiting and rollback cycle near the central blocker and times out after 65 s. The rendered path is reconstructed from recorded execution states; only light visualization smoothing is applied, without altering the logged topology.

Scenario	Failure	Collision	Unresolved recoveries	Max wait p75 [s]	Dominant mode
Corridor Trap	0.0%	0.0%	0.00 ± 0.00	7.1	none
Crossing Humans	0.0%	0.0%	0.00 ± 0.00	1.1	none
Random Crowd	0.0%	0.0%	0.00 ± 0.00	2.5	none
Structured Crowd	0.0%	0.0%	0.00 ± 0.00	1.9	none
Stress Crowd*	100.0%	0.0%	0.22 ± 0.41	11.1	deadlock
Permanent Blocking**	100.0%	0.0%	0.00 ± 0.00	76.1	deadlock

Table 5: Failure-oriented diagnostics derived from logged runs. The starred stress row is a logged adversarial crowd suite used to expose safe failure; the double-starred permanent-blocking row is an out-of-regime diagnostic used to illustrate behavior when admissible-dynamics assumptions are violated.

objective here is not broader distributional coverage but deterministic, inspectable execution with reproducible logs and a proof aligned to the implemented controller stack. In dense crowds, the logged random-crowd suite shows successful traversal with elevated waiting and recovery activity rather than silent regression. Finally, safety should be read as *runtime enforcement* in the implemented simulator through projection and rollback, not as a model-free certificate against arbitrary unmodeled disturbances or adversarial agents.

The nominal per-step execution cost scales approximately as

$$\mathcal{O}(H + O + P), \tag{18}$$

where H is the number of humans, O the number of static obstacles or zones, and P the number of guide polyline segments. Intermittent guide rebuilds use graph search and can scale as $\mathcal{O}(G \log G)$ in the number of guide nodes

G.

7 Conclusion

We presented a deterministic navigation system with execution-level progress enforcement, runtime safety projection, and rollback-based correction. The resulting closed loop is safety-enforced via runtime projection and rollback, deadlock-free under safe-feasible conditions, and convergent only under the stated admissible conditions. The main message is execution-level correctness: the proof is aligned with the motion the robot actually executes, and the reported evidence is generated directly from reproducible logged runs.

References

[1] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research*, pp. 3–19, Springer, 2011.

[2] D. Helbing and P. Molnár, "Social force model for pedestrian dynamics," *Physical Review E*, vol. 51, no. 5, pp. 4282–4286, 1995.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[4] Y. F. Chen, M. Everett, M. Liu, and J. P. How, "Socially aware motion planning with deep reinforcement learning," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1343–1350, 2017.